

SGP: Sistema Gestione Personale

Studente: *Matteo Giorgio* – Classe: *5^a B Informatica* – Anno Scolastico: *2025/2026*

Repository github: <https://github.com/zMqTTew/Sistema-di-Gestione-Personale-SGP->

1. Introduzione e motivazioni dell'aggiornamento

L'idea alla base di questo progetto nasce da un'esigenza concreta e quotidiana. Utilizzando intensamente il computer per lo studio e lo sviluppo, mi sono reso conto di quanto tempo utile perdessi a causa del disordine: file scaricati, documenti sparsi e cartelle caotiche rendevano spesso le sessioni di lavoro frustranti. Inoltre a questo si aggiungeva l'idea di gestire in modo sicuro le mie credenziali d'accesso ai vari servizi web, senza dipendere da soluzioni esterne di cui non ho il pieno controllo.

Invece di affidarmi a software commerciali di terze parti, ho deciso di sfruttare le competenze acquisite durante il mio percorso scolastico per realizzare un software mio, personale, che potessi utilizzare concretamente. Per farlo ho voluto fare un passo oltre il programma scolastico, documentandomi e integrando nel progetto librerie specifiche che non avevamo mai applicato in classe (come *CustomTkinter* per l'interfaccia grafica oppure *Cryptography* per la sicurezza).

Dopo aver usato per alcuni mesi la prima versione del programma, mi sono però accorto che la scelta di affidarmi a un semplice file JSON locale per il Password Manager, per quanto comoda da implementare, aveva un limite enorme: le credenziali restavano legate a un solo computer. Da qui è nata l'idea di trasformare SGP da un'applicazione mono-utente con salvataggio locale a un sistema multi-utente con un database centralizzato, raggiungibile da qualsiasi dispositivo collegato alla rete VPN.

2. Obiettivi del software

Rispetto alla 1.0, gli obiettivi principali di questo aggiornamento sono stati:

1. **Accesso multi-dispositivo:** permettere di accedere allo stesso archivio di password da qualsiasi PC della rete VPN, senza dover sincronizzare manualmente alcun file.
2. **Gestione multi-utente:** introdurre un sistema di registrazione e login che permetta a più persone di avere un proprio spazio separato e protetto.
3. **Maggiore robustezza della sicurezza:** sostituire la semplice combinazione hash + salt fisso della versione 1.0 con un sistema di derivazione della chiave più solido, con salt univoco per ogni utente.
4. **Monitoraggio hardware più completo:** estendere il modulo di monitoraggio risorse aggiungendo anche le statistiche della GPU.

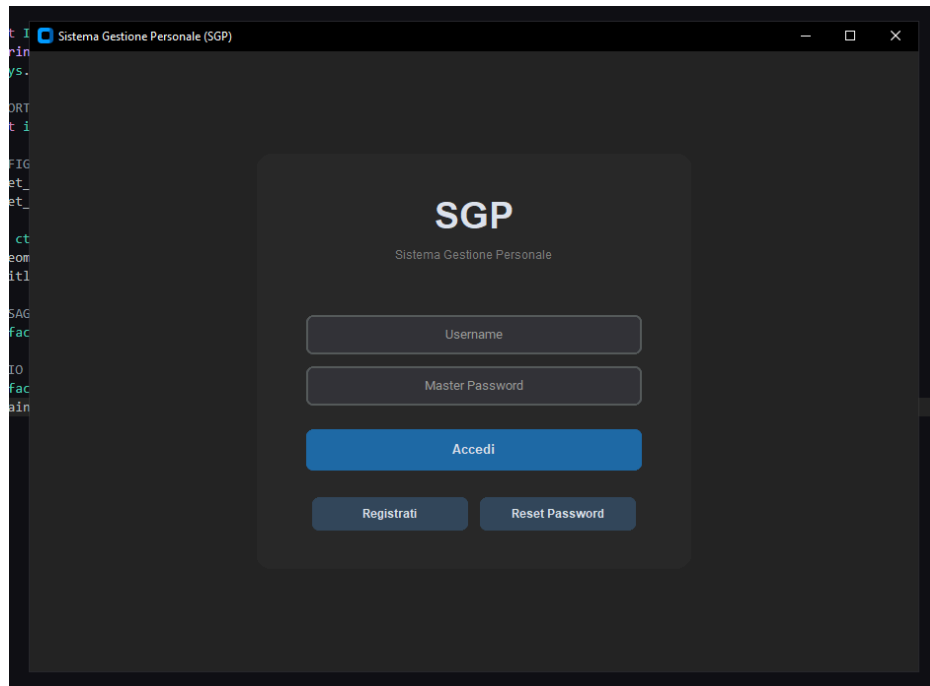
3. Architettura del software e scelte tecniche

L'applicazione è stata suddivisa in quattro moduli distinti e interconnessi:

- **main.py:** punto di ingresso del programma, verifica la presenza delle librerie necessarie, inizializza la finestra principale e avvia il loop dell'applicazione.
- **interfaccia.py:** gestisce tutta la parte grafica: le schermate di login, registrazione, reset password, il menu principale e lo switch dinamico tra i vari pannelli (File Manager, Password Manager, Stato PC).
- **sicurezza.py:** il modulo che ho riscritto più radicalmente rispetto alla 1.0. Non gestisce più un file JSON locale, ma dialoga con un database PostgreSQL remoto: si occupa di

autenticazione, registrazione di nuovi utenti, derivazione della chiave di cifratura e delle operazioni di salvataggio/recupero delle credenziali cifrate.

- **gestore_file.py**: contiene le regole di scansione delle directory e il riordino dei file in base all'estensione.



4. Dal file JSON al database remoto

Il cambiamento più significativo della versione 2.0 riguarda il modo in cui i dati vengono salvati e protetti. Nella 1.0 avevo scelto consapevolmente di evitare database e server locali, per mantenere l'applicazione leggera e portabile. In questa versione ho invece deciso di introdurre un vero database relazionale PostgreSQL: il database non è ospitato su un servizio cloud di terze parti, ma gira direttamente sul mio PC fisso di casa, raggiungibile dagli altri dispositivi tramite una rete privata virtuale (VPN).

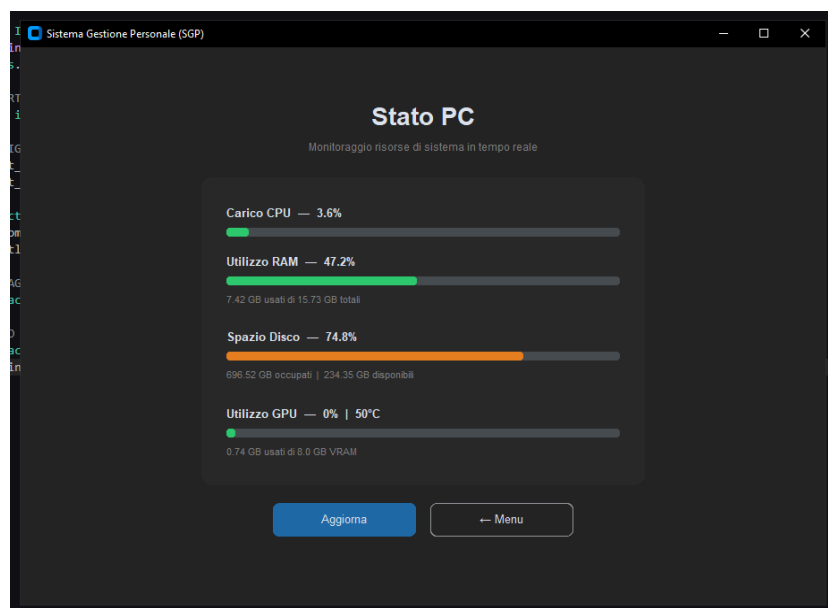
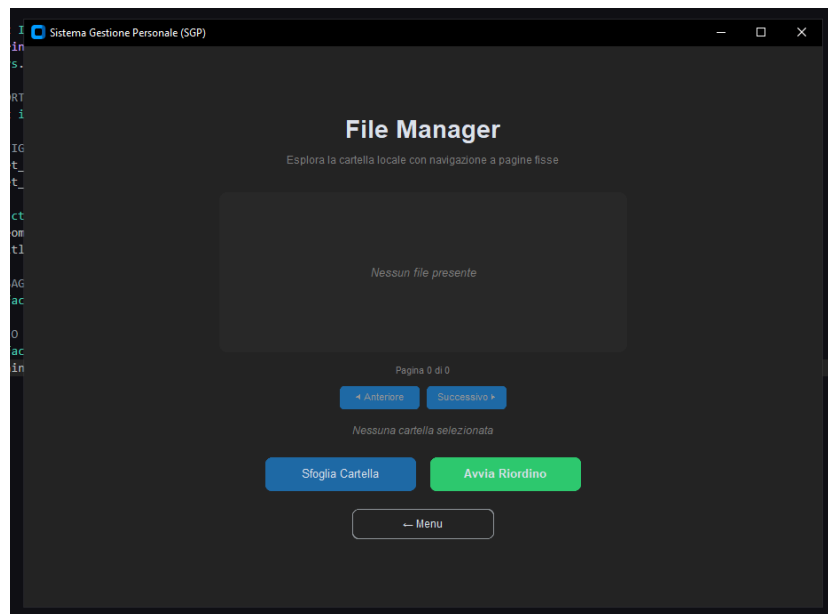
A livello di sicurezza, ho mantenuto e rafforzato le tre barriere già presenti nella 1.0:

1. **Protezione della Master Password**: la password non viene mai salvata in chiaro. Viene generato un hash SHA-256 della password unita a un salt, e confrontato con quello salvato nel database al momento del login.
2. **Salt individuale per ogni utente**: a differenza della 1.0, dove il salt era un parametro fisso dell'applicazione, ora ogni utente registrato ha un proprio salt casuale generato con `os.urandom`, rendendo molto più difficile un attacco che sfrutti tabelle precalcolate (rainbow table) su più account.
3. **Derivazione della chiave con PBKDF2HMAC**: la chiave di cifratura simmetrica usata per il vault non è più ricavata in modo semplice, ma derivata dalla master password e dal salt dell'utente attraverso 100.000 iterazioni dell'algoritmo PBKDF2HMAC con SHA-256, rendendo computazionalmente costoso un eventuale tentativo di forza bruta.
4. **Cifratura simmetrica Fernet (AES-128)**: le singole credenziali vengono cifrate prima di essere scritte nel database, in questo modo anche un accesso non autorizzato al database stesso non sia sufficiente a leggere le password in chiaro.

5. Alcune fasi dello sviluppo

Il percorso di sviluppo non è stato lineare e mi ha costretto a mettermi in gioco più volte:

- **Fase 1: Migrazione della logica di sicurezza.** Ho dovuto riscrivere da zero il modulo `sicurezza.py`, passando dalla logica di lettura/scrittura su un dizionario JSON alla logica di interrogazione di un database relazionale con query SQL.
- **Fase 2: Configurazione della rete e della VPN.** La parte più nuova per me è stata configurare l'accesso remoto al database del PC di casa. Ho dovuto documentarmi sul funzionamento di una VPN privata per collegare in sicurezza i miei dispositivi, senza dover esporre il database direttamente su Internet.
- **Fase 3: Gestione degli errori di connessione.** Introducendo un componente di rete, è comparsa una categoria di errori che nella 1.0 non esisteva: la possibilità che il server non sia raggiungibile, e quindi ho dovuto gestire con cura questi casi nell'interfaccia, distinguendo chiaramente un errore di rete da una semplice password sbagliata, in modo da non confondere l'utente con un messaggio non corretto.
- **Fase 4: Estensione del monitoraggio hardware.** Ho ripreso uno degli sviluppi futuri lasciati in sospeso nella 1.0, aggiungendo al modulo Stato PC anche il monitoraggio della GPU tramite la libreria `pynvml`, gestendo il caso in cui la libreria o l'hardware non siano disponibili (ad esempio su portatili senza GPU dedicata), mostrando in quel caso un'indicazione esplicita di GPU non rilevata anziché un errore.



6. Conclusioni e sviluppi futuri

Realizzare questa seconda versione di SGP mi ha permesso di affrontare problemi più vicini a quelli di un'applicazione reale multi-utente, distribuita su più dispositivi, piuttosto che a un semplice programma locale. Mettere in comunicazione un'interfaccia grafica, un database remoto e un sistema di cifratura mi ha fatto capire meglio quanto la sicurezza non sia un singolo accorgimento, ma il risultato di più livelli che lavorano insieme.

Anche questa versione non è comunque definitiva. Per il futuro confermo le idee già indicate nella versione 1.0 e ancora da realizzare, come l'inserimento di un generatore automatico di password robuste basato sulla libreria `secrets`, così da non dover più affidarmi alla fantasia dell'utente per scegliere chiavi sufficientemente complesse.

Per quanto riguarda invece il monitoraggio hardware, l'obiettivo che mi ero posto nella versione precedente per la GPU è stato in parte raggiunto: il modulo Stato PC mostra ora anche il carico, l'utilizzo e la temperatura della GPU, sfruttando la libreria `pynvml` per le GPU NVIDIA.